Wireshark Network Traffic Analysis

Capture and Analyze Network Traffic

Prepared by Vimal Raj R ·

Cyber Security Intern · April 2026

Executive Summary

Purpose

This report documents the capture and analysis of live network traffic using Wireshark, focusing on observing real-time packet flow and identifying common network protocols.

Key Activities

- Browsed websites and sent network requests to generate traffic
- Applied display filters to isolate DNS, HTTP, ICMP, and TCP packets
- Inspected packet-level details and communication patterns
- Exported captured data as a `.pcap` file for documentation

> Packet inspection is a foundational skill in cybersecurity — understanding normal traffic is the first step to detecting anomalies.

Objectives & Tools

Learning Objectives

→ Capture Live Traffic

Record packets flowing through an active network interface in real time.

→ Identify Protocols

Recognize DNS, HTTP, ICMP, and TCP traffic using Wireshark filters.

→ Analyze Communication

Inspect packet-level details to understand how data travels across networks.

Tools Used

Wireshark

Packet capture & analysis

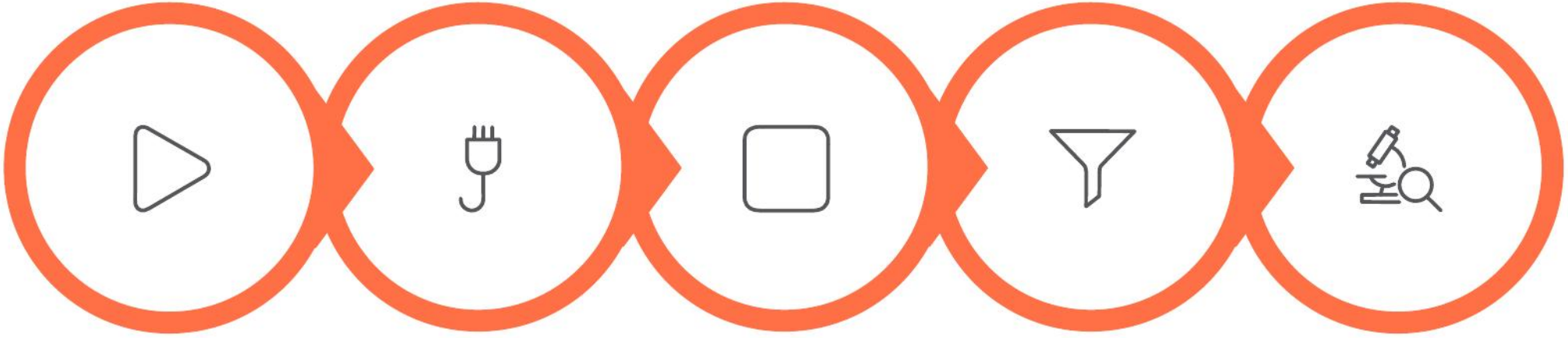
Web Browser

HTTP traffic generation

Command Line

Ping & HTTP requests

Methodology



Start Capture

Generate Traffic

Stop Capture

Apply Filters

Analyze Packets

Each step builds on the previous — starting from raw capture through to structured analysis and export — mirroring a real-world SOC analyst workflow.

Traffic Generation

The following actions were performed to produce a diverse mix of network traffic for analysis:

HTTP Traffic

Opened websites in a browser to generate GET requests and server responses over port 80.

DNS Lookups

Navigating to domains triggered DNS queries, resolving hostnames to IP addresses via UDP port 53.

ICMP Ping

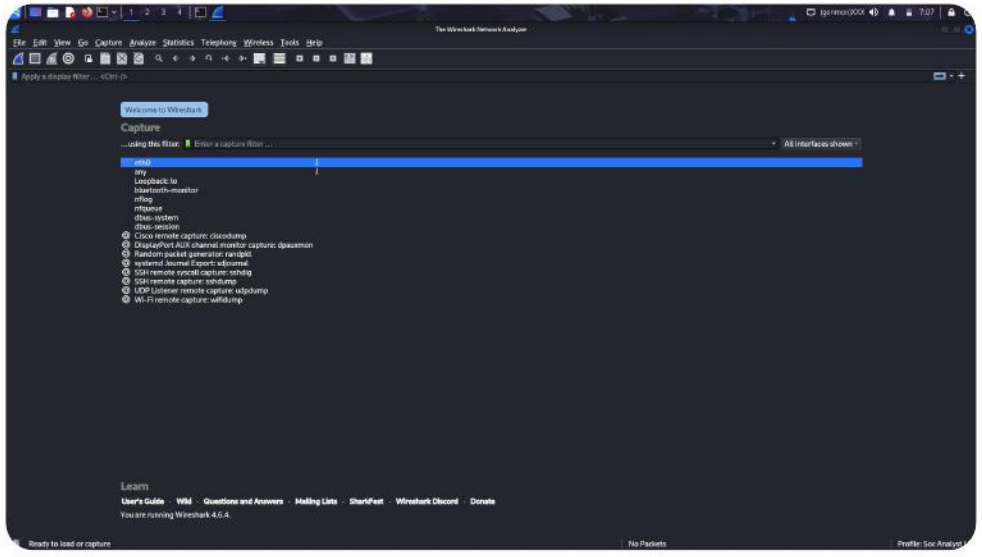
Used the `ping` command to send echo requests and capture corresponding echo replies.

TCP Handshake

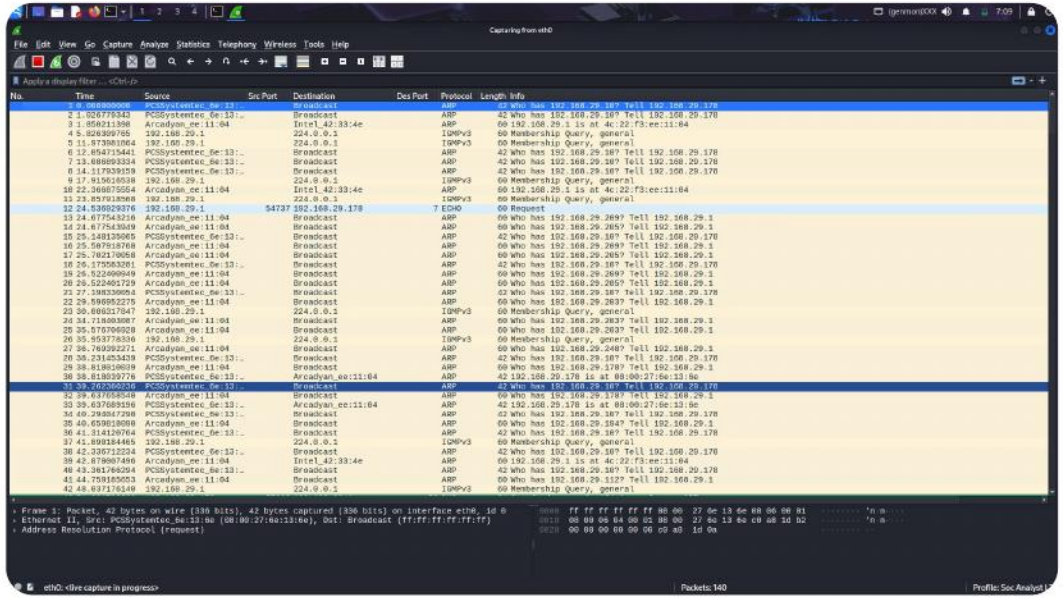
Observed the three-way handshake (SYN → SYN-ACK → ACK) during connection establishment.

SCREENSHOT 1 & 2

Wireshark Capture Interface & Live Traffic



Wireshark start screen showing available capture interfaces.

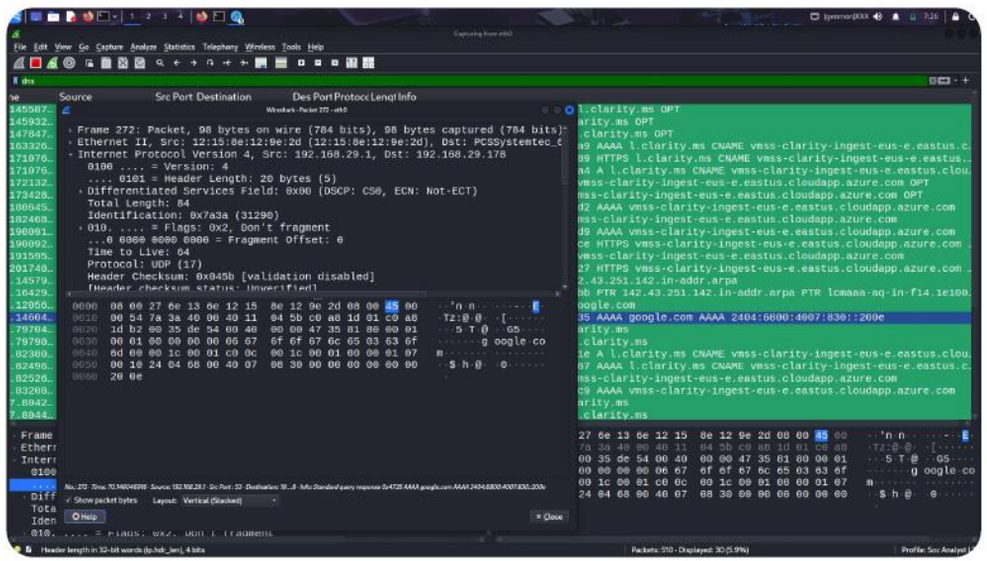


Active capture session showing mixed protocol traffic — ARP, DNS, ICMP, and more — all generated during the test.

DNS & HTTP ANALYSIS

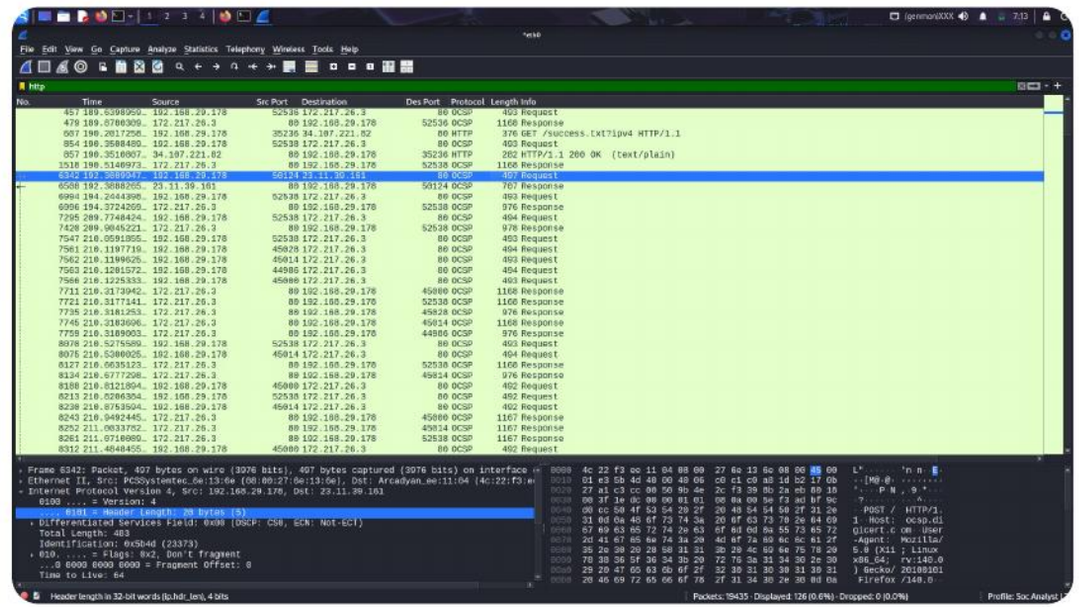
Protocol Filtering: DNS & HTTP

DNS — Filter: dns



DNS query and response packets visible — domain names resolved to IP addresses over UDP port 53.

HTTP — Filter: http



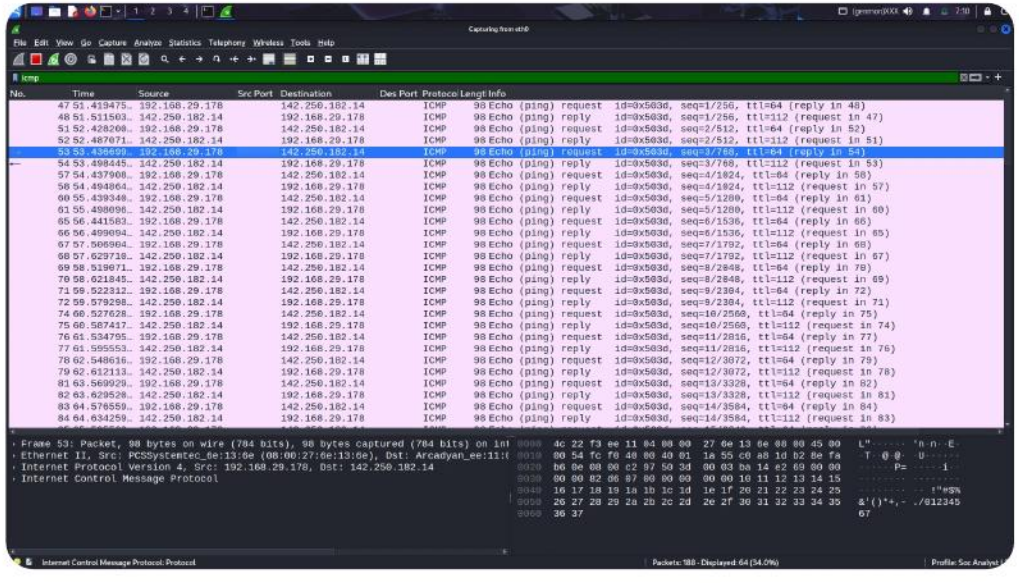
HTTP GET requests and 200 OK responses captured — unencrypted web traffic fully visible at the packet level.

⚠ HTTP traffic is unencrypted. Sensitive data transmitted over HTTP is visible in plaintext within packet captures.

ICMP & TCP ANALYSIS

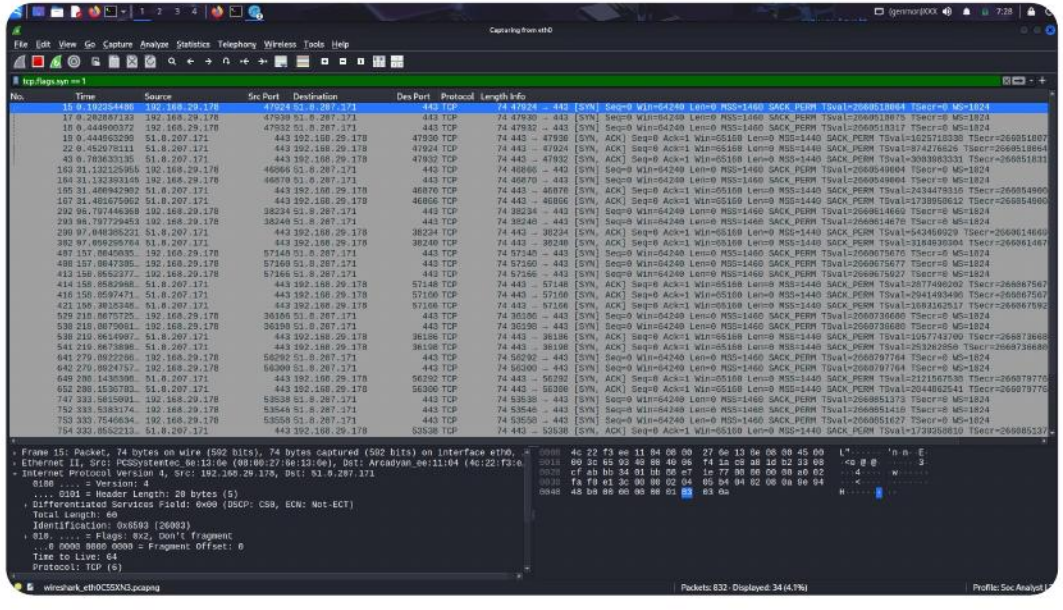
Protocol Filtering: ICMP & TCP

ICMP — Filter: icmp



Echo requests and replies from ping command — sequence numbers confirm successful round-trips.

TCP — Filter: tcp.flags.syn == 1

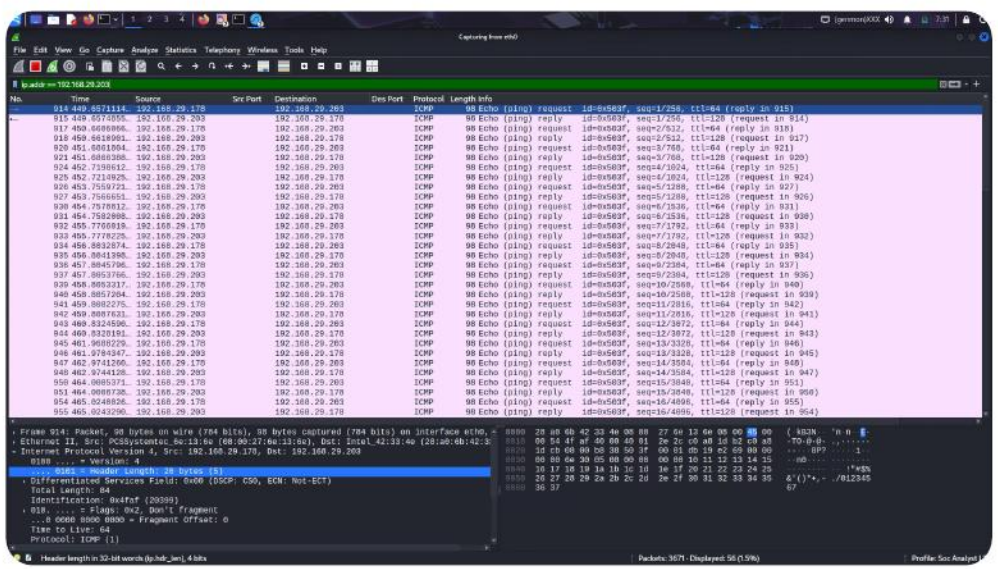


SYN packets isolated — each represents the start of a new TCP connection. High SYN counts can indicate port scanning or SYN flood attacks.

ADVANCED ANALYSIS

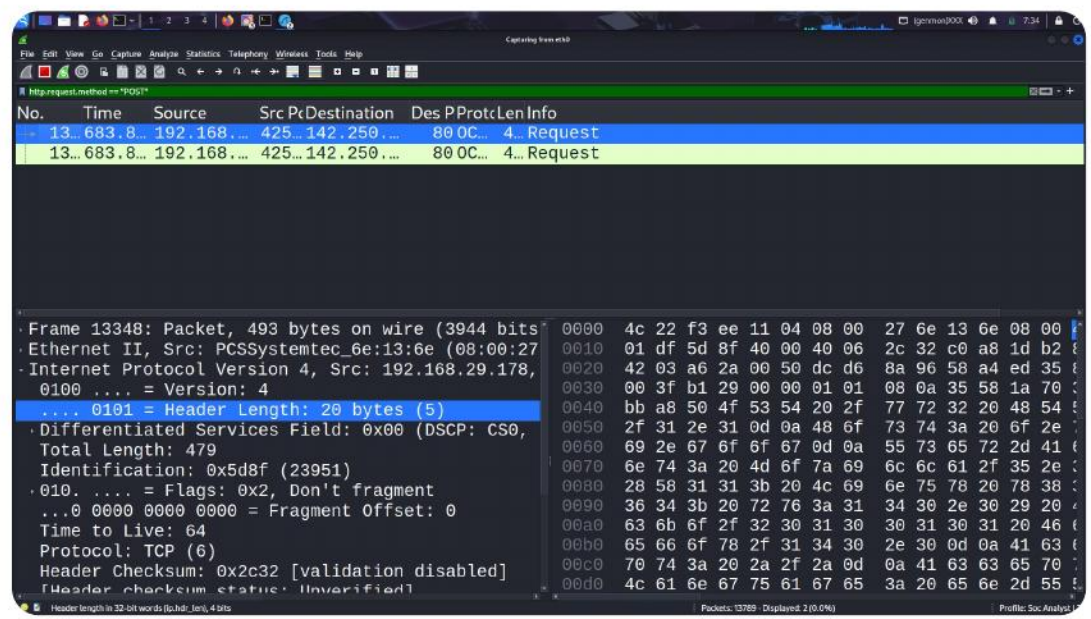
Advanced Filters: IP & POST Requests

Local Traffic — Filter: `ip.addr == [host IP]`



Filtered view shows all traffic to and from the local machine IP — useful for isolating host-specific activity.

HTTP POST — Filter: `http.request.method == "POST"`



POST requests captured — form data submitted over unencrypted HTTP is fully readable, underscoring the risk of non-HTTPS sites.

Key Takeaways

Wireshark = Visibility

Real-time packet capture gives analysts granular insight into every layer of network communication — from Ethernet frames to application-layer payloads.

Filters Are Essential

Display filters like `dns`, `http`, `icmp`, and `tcp.flags.syn==1` allow rapid isolation of relevant traffic from thousands of captured packets.

HTTP Is a Risk

Unencrypted HTTP exposes GET/POST data in plaintext. Always prefer HTTPS — and flag unencrypted traffic as a finding during assessments.

Foundation for Threat Detection

Understanding normal traffic baselines is critical for identifying anomalies such as port scans, data exfiltration, or C2 beaconing in real-world SOC environments.

✓ Capture file exported as `Networks_Analysis.pcap`. All objectives met.